

What is object oriented programming?

Object-oriented programming (OOP) is a programming paradigm built around objects: self-contained units that bundle data (attributes) and behavior (methods). It helps developers model real-world concepts in code, reuse components, and manage complexity in large software systems.

Key Concepts

- **Classes** - Blueprints that define what data and behaviors objects will have.
- Example: A “Car” class defines color, brand, and methods like start or brake.
- **Objects** - Actual instances created from classes.
- Example: A red Toyota is an object created from the Car class.
- **Encapsulation** - Hiding internal details and exposing only what’s necessary.
- Protects data using access controls like private fields.
- **Abstraction** - Showing only essential features while hiding implementation details.
- Makes code simpler and easier to use.
- **Inheritance** - Creating new classes that reuse and extend existing ones.
- A “SportsCar” class can inherit from “Car”.
- **Polymorphism** - Different objects responding to the same command in different ways.
- Example: draw() behaves differently for a Circle vs. a Rectangle.

What Does OOP Do In Python?

1. **It lets you create your own data types (classes).**

Python already gives you built-in types like int, str, and list. With OOP, you create custom types that behave exactly how you want.

2. It bundles data + behavior together.

A class can store **attributes** (data) and **methods** (functions).

3. It creates objects (instances) you can use.

Objects are real things created from the class.

4. It hides complexity (encapsulation).

You can protect internal details so users only interact with what matters.

5. It lets classes reuse code (inheritance).

A new class can extend an old one.

6. It allows different objects to behave differently (polymorphism)

OOP in Python helps you

- Organize code into reusable pieces
- Model real-world things
- Keep code clean and manageable
- Build larger programs without losing control

Application Examples

Desktop Applications

- Microsoft Office Suite (Word, Excel, PowerPoint)
- Adobe Photoshop
- Eclipse IDE / Visual Studio

Web Applications

- Facebook
- Gmail (Java, JavaScript)
- Spotify (Python + Java)

Mobile Applications

- Instagram
- Uber
- Whatsapp

Games

- Minecraft (Java)
- Unity Engine (C#)
- The Sims

Comparison Between Python and Java

1. Typing

- **Java:** Statically typed: you must declare types.
- **Python:** Dynamically typed: types decided at runtime.

2. Syntax

- **Java:** More strict, more code, everything inside a class.
- **Python:** Cleaner, shorter, more flexible.

3. Encapsulation

- **Java:** Real access modifiers (public, private, protected).
- **Python:** Uses naming conventions (_var, __var), not enforced.

4. Inheritance

- **Java:** Single inheritance + interfaces.
- **Python:** Multiple inheritance allowed.

5. Polymorphism

- **Java:** Checked at compile time + runtime.
- **Python:** Mostly runtime because of dynamic typing.

6. Performance

- **Java:** Faster (JIT compilation).
- **Python:** Slower unless using C-based libraries.